BizTrack — Five Workstreams

30 August 2026 · commit `9a33fe6` on `main`

Live: https://apartments-ever-bay-climb.trycloudflare.com

Contents

What this is

Five pieces of work shipped together in one commit, because they all touch the same thing: what an applicant is asked, and when.

1. **Payment first.** The six LGU clearances move back out of the wizard. You apply for your business permit, you pay for it, and the clearance stage opens after that — each clearance its own fee, its own office, its own errand.
2. **Renewal.** A renewal now names the permit it renews, or says explicitly that there is none to name. The permit chain became a stored fact.
3. **Amendment.** The same, for amendments — plus an audit of a feature that turned out to be already built, and two gaps in it that we are reporting rather than half-fixing.
4. **Office forms.** An audit against the rule *each office sheet asks only what its paper form needs, minus what the wizard already holds.* One real duplicate found, and it was billing a different number than it displayed.
5. **Messaging.** A conversation now has an addressee. It did not before — the table had no column for one.

How to read this

Each section leads with what changed and why, in plain language, before any code detail. Every factual claim names the file and line it can be checked against.

Where we guessed, the guess is written down in the format used throughout `docs/questions-for-malabon.md`: the question, why it matters, and what we assumed meanwhile. If an answer from City Hall contradicts an assumption, we change the system. Nothing here is defended.

Where what we found in the code differed from what we had been told about it, the report says so and states what is true. Those corrections are collected in the last section.

What was checked, and how

Check	Result
Backend test suite (`php artisan test`)	**832 tests passed, 10,582 assertions, 0 failed** — 85 seconds
Browser test suite (`playwright test --workers=1`)	**158 passed, 1 skipped, 0 failed** — 9.4 minutes, over 159 declared cases in 13 files
TypeScript (`npx tsc -b --force`)	exits **0**
Live demo tunnel	responds **200**

Both suites were re-run for this report rather than quoted. The browser suite was run on its own isolated stack — its own copy of the register, its own ports — so that it could not disturb the live demo.

One number differs from what we had been told, and the difference is worth understanding rather than hiding. The figure we were given was 155 passed and 4 skipped; we measured 158 passed and 1 skipped, with the same total of 159 and no failures either way. Several of these tests skip *themselves* when the register they are pointed at holds no suitable filing to work on — the one that skipped for us was `office-`

`scoping.spec.ts:854`, *“offices on one filing do not read each other’s conversation with the applicant”*, which needs a filing routed to two offices. So the pass/skip split is a property of the copied register on the day, not of the code. What matters, and what is stable, is that nothing failed.

Two notes on the test commands, both learned the hard way.

`tsc --noEmit` must never be used here. The root `tsconfig.json` is `{"files": [], "references": [...]}`, so `--noEmit` checks **zero files** and always passes. `tsc -b --force` is the command that actually builds the two referenced projects.

The browser suite must be run serially. `playwright.config.ts` defaults to `workers: 4` outside CI, and four workers against one SQLite file produce contention failures that pass in isolation — which is the worst kind of test result, because it is neither a pass nor a reproducible failure.

1. Payment first: the ordering reversal

What changed, and why

The six LGU clearances — Zoning, Sanitary, Fire, Environmental, Occupancy, Market — used to be chosen *inside* the application wizard, before submission, with one Tax Order of Payment covering the business permit and every clearance together. That has been reversed.

The wizard is now the business permit alone. You submit it, you pay for it, and only then does the clearance stage open. From that point each clearance is its own transaction: pressing **Apply** adds that office's fee to a running balance and routes the filing to that office at that moment, and no permit is released while the balance is outstanding.

```
WIZARD — the business permit application
  1 Data Privacy Consent      4 Documentary Requirements
  2 Location & Zoning         5 Business & Tax Profile
  3 Business Information      6 Review & Submit
        |
  SUBMITTED  ->  Tax Order of Payment #1 (business permit)  ->  PAID
        |
LGU CLEARANCES — the stage opens here
  Apply  -> that office's form opens, its fee joins the balance,
            the filing is routed to that office now
  Submit -> upload the copy you already hold. No fee, no form.
        |
  BALANCE DUE must reach zero before any permit is released
```

The wizard's six steps are exactly that list — `ApplyWizard.tsx:69` and `:141`:

```
const BASE_PHASES: BasePhase[] = ['privacy', 'address', 'business', 'documents', 'fees',
        'review']
```

There is no `permits` phase and no office-form sheets inside the wizard; both left with the reversal.

Why reverse a decision that was already made

Because the reason for the earlier reversal has not gone away, and neither has the reason for this one. Both directions are defensible and both cost something:

- **Clearances chosen inside the wizard** is simpler to build. One assessment, one payment, one gate. But an applicant has to decide, at the counter, which of six clearances they need before they have spoken to any of the six offices.
- **Clearances after payment** matches what actually happens at City Hall: you get your business permit application in and paid for, and the clearances are separate errands with separate offices and separate fees. The cost is that it needs an accruing balance, a second payment, a release gate, and a stage that has to be locked and explained before it opens.

This is the arrangement the client asked for, so this is what is built. It is worth saying plainly that this is the second reversal, and that the file recording the design was written for the first one.

A documentation defect we did not fix. `docs/clearances-after-payment.md` is the spec this work was built against, and roughly fifteen code comments written in this commit cite it as the live specification — two of its rules are quoted verbatim in `PermitFees` and `WorkflowService`. But the file still opens with `# SUPERSEDED — 4 August 2026` and points the reader at `docs/clearances-before-payment.md`, which describes the design we have just undone. Neither file was touched by this commit. A reader following the pointer would read the wrong spec and conclude the code was wrong. Both headers need correcting; we have left them as found rather than edit documentation while writing a report about it.

The three bugs this design grows

The superseded spec predicted where this arrangement produces bugs. All three appeared, and all three are fixed.

1. The unpayable balance, one status over

The payment endpoint was already open to filings past the first payment. What was not right was its definition of "closed". It used `ApplicationStatus::isTerminal()`, and that enum counts **Approved** as terminal alongside Rejected and Cancelled.

Approved is exactly the state a filing is in when a business comes back in October to add a food line and needs a sanitary clearance it did not need in January. So that applicant could apply for a clearance, see the fee, owe the fee, be blocked by the fee — and be refused when they tried to pay it.

Narrowed to the two statuses that really are closed — `PaymentController.php:99–109`:

```
$closed = in_array($application->status, [ApplicationStatus::Rejected,
        ApplicationStatus::Cancelled], true);
```

2. The release gate locking officers out of their own work

The rule "no permit while money is owed" was first put in one place: `approveAndIssue()`. That method is called from the automatic paths too — when the last office approves its review, and when the last inspection is recorded. So the refusal was thrown *inside the transaction that had just marked that office's own assignment completed*. An office could not finish its own review because of somebody else's unpaid bill, and the transaction rolled back its approval with it.

The fix is to split the two audiences for that rule:

- A **direct caller** — an officer pressing the button that issues the permit — gets a hard `ValidationException`, keyed on `balance_due` (`WorkflowService.php:1102–1110`): *"This application still owes ₱… on the clearances applied for after payment. Permits are released once the balance is settled."*
- The **automatic paths** get a quiet "not yet". `isFullyCleared()` (`WorkflowService.php:744–809`, balance branch at `:791–793`) simply returns false, so the office's approval is recorded, nothing is rolled back, and the filing waits.
- `releaseIfSettled()` (`WorkflowService.php:459–464`) picks it up when the money lands. Its one call site is `onPaymentCompleted()` at line 344.

`isFullyCleared()` has four call sites — the classifier, `releaseIfSettled`, `afterReviewProgress` and `recordInspection` — which is the point: every route to release asks the same question.

A correction to how this was described to us. Relative to the immediate parent commit, the gate did not exist at all to be misplaced; the parent's `approveAndIssue` docblock said in terms *"There is no balance check here, and there should not be one"*, because under the one-payment design there was never a balance. The misplaced-gate bug belongs to the pre-4-August build of this same design. What this commit does is re-add the gate, already split. That is the same lesson learned once rather than twice, but it is not a bug that was live in the code a week ago, and the report should not imply it was.

3. Office sheets that could be billed for but never filled in

An office sheet is editable while the application is a draft or has been returned. Under the previous ordering that was the whole story, so the second edit window was deleted as dead code.

Under this ordering it is not dead: a clearance applied for after payment first becomes reachable on a filing that is already under review, and the applicant would have been billed for a form they could never open.

The window is restored, and it is now per-sheet rather than per-filing — `OfficeFormController.php:198–214`. Draft or Returned, yes; Rejected or Cancelled, no; otherwise the sheet is open while *that clearance's own assignment* is not yet completed. The refusal explains itself:

> This form can no longer be edited. Office forms are open while the application is a draft or has been returned to you, and while a clearance you applied for is still waiting on its office.

Covered by `it('lets the applicant fill in the sheet for a clearance applied for after payment')`, `ClearanceStageTest.php:880`.

One design detail worth explaining

The stage unlocks on a fact from the **payments ledger**, not on the application's status — `PermitFees::hasClearedPayment()`, `PermitFees.php:91–96`:

```
return $application->payments()->where('status', PaymentStatus::Completed->value)-
        >exists();
```

Only a `Completed` payment counts; pending, failed and refunded do not.

Status alone would get two ordinary cases wrong, in opposite directions:

- A filing **rejected while still at `pending_payment`** never paid. Asking the status would find a terminal state and might unlock or lock for the wrong reason; asking the ledger finds nothing settled, and the stage stays shut. (Belt and braces here: the closed-status list catches it too. Test at `ClearanceStageTest.php:337`.)
- A filing **returned to the applicant after payment** has a status that looks unfinished but a ledger that says paid. The stage stays open, which is right — the applicant is fixing something, not starting again. Test at `ClearanceStageTest.php:299`.

The ledger is the record of what actually happened. The status is a summary of where the filing is. For "has this been paid for", only one of those is authoritative.

What we assumed, and whether the code enforces it

Seven decisions were taken to keep moving. Six are enforced by code; one is only written down. The distinction matters, so here it is honestly:

Assumption	Status
The stage unlocks on the **first payment clearing**, not on submission	Enforced — `ClearanceService::isUnlocked()` (`:288–295`); the controller returns 422 with the locked reason as its body
A clearance may be applied for **after the permit is released**	Enforced — `Approved` is deliberately absent from the closed list, and the payment fix above makes the resulting balance payable
A **rejected clearance** does not kill the business permit	**Stated only, and currently unreachable.** `AssignmentStatus` has no `Rejected` case at all — it is Pending, InProgress, Completed, Returned. The clearance API still lists `'rejected'` as a possible state while nothing in the system can produce one. This is checklist item 80 and question A2, still open
Fee gating untouched — the Fire Code fee and the sanitary inspection fee stay gated on their clearance	True: this commit changes neither `FeeCalculator.php` nor any revenue-code data file. 8 rules are gated on `FSIC` , 21 on `SANITARY`
No refund when a clearance is withdrawn	Enforced — `PermitFees::balance()` floors at zero. Test at `ClearanceStageTest.php:566`
The gate is **per filing**, not per permit	Enforced — the balance is tested once, then every permit type is minted. The payments ledger does not attribute a payment to a permit type, so a per-permit gate has nothing to compute from
Re-assessment overwrites an officer-adjusted assessment	Enforced as a consequence, not by intent: `assessFees()` uses `updateOrCreate` on `application_id` , and an officer's adjustment writes the same single row. No test asserts it

Two of those deserve a footnote.

"Applying after release is allowed by the data model but not surfaced on the screen" is what the code comment says, and it is no longer true. The link to the clearance stage renders for every non-draft status including Approved (`ApplicationDetailPage.tsx:670–685`), and the stage renders unlocked. The behaviour is right and the comment is stale.

The fee-gating decision is the one we most want overturned by an answer, not by us. Uploading a clearance you already hold escapes the Fire Code fee and the sanitary inspection fee, because those fee rules are gated on the clearance being applied for. That is arguably wrong under RA 9514. Changing what a citizen is charged, on our own reading of a statute, is not a call a student project should make without BPLO. It is written down rather than quietly fixed.

The open risk, stated plainly

A clearance can be withdrawn after its fee has been paid, and the money has nowhere to go.

The withdraw path (`ClearanceController::unapply()`, lines 95-126) checks ownership, that the stage is unlocked, that the clearance is priceable, that it was applied for, that no permit has issued, and that the office has not yet acted. **It does not check whether the fee was paid.** So a clearance whose office is still `pending` can be withdrawn after payment.

One correction to how this was described to us: the balance does **not** go negative. `PermitFees::balance()` floors at `0.0`, so the filing is left in silent credit rather than showing a negative figure. The test that locks this in (`ClearanceStageTest.php:566–581`) asserts exactly that — after apply, pay, withdraw, `total_paid − total_assessed` is ₱735.00 and `balance_due` is ₱0.00.

On screen the applicant sees Paid greater than Assessed and "Balance due ₱0.00" rendered in green, as though nothing were owed in either direction. No wording anywhere names the credit or offers it back.

This is newly reachable — under the one-payment design there was no moment between paying and withdrawing. It needs a BPLO answer: refund, credit against the next clearance, or forfeit. Refunds are not modelled at all, so whichever answer comes back is a piece of work, not a setting.

On the screen

The lock reason is printed verbatim. `ClearanceStagePage.tsx:869–887` renders `meta.locked_reason` and nothing else. The previous version put a fixed heading above it — *"These can no longer be changed"* — and that heading was deleted because the stage now has two opposite reasons for being locked. Not-yet-open and closed-after-rejection are both locks, and a single heading is wrong about one of them. The server writes six distinct sentences (`ClearanceService::lockedReason()`, lines 307-328), including:

- Draft — *"Finish and submit this application first, then settle the Tax Order of Payment for your business permit. The six LGU clearances open here the moment that payment clears."*
- Rejected — *"This application was not approved, so no further clearances can be applied for under it. File a new application if you still need these clearances."*
- The catch-all — *"The LGU clearances open once the first payment on this application has cleared. Ours shows nothing settled yet — contact the BPLO if you have already paid."*

The same strings are the body of the 422 when a locked write is attempted, so the screen and the API cannot disagree about why.

The ledger shows while the stage is locked too (lines 913-941, rendered outside the locked branch). Assessed, paid, balance due. This looks like a mistake and is not: when the stage is locked *because you have not paid*, the amount you have to pay to unlock it is the single most useful thing on the page.

`fee_preview` is back on each clearance card (line 1085-1087). It had gone dead — in the parent commit the `feeAmount` helper was exported and called from nowhere in the entire frontend. Under the one-payment design that was survivable, because there was a later screen showing the Tax Order of Payment before you committed to it. There is no such screen now: Apply spends money immediately, so the card has to say how much.

Tests

The claim we were given was “ten tests renamed, seven new”. The real numbers, from pairing every test title before and after:

File	Before	After	Renamed	New
`api/tests/Feature/ClearanceStageTest.php`	35	42	12	7
`web/e2e/clearances.spec.ts`	13	14	4	1

So **16 renamed and 8 new**, not 10 and 7. (One PHP pairing is a judgement call; counting it as a deletion plus an addition gives 11 renamed and 8 new in that file.) “Seven new” is right for the PHP file taken alone, which is probably where the figure came from.

The renames are the interesting part, because each one is a sentence that used to state the opposite rule:

- `opens the stage while the application is still a draft` → `opens the stage the moment the first payment clears`
- `carries no ledger in meta, because a draft owes nothing` → `carries the ledger in meta, because applying raises a balance`
- `routes every chosen clearance to its own office when the payment clears` → `routes a clearance to its own office the moment it is applied for`
- `issues every permit once the offices sign off, with no balance to settle` → `releases no permit while the clearance balance is outstanding`

That last pair runs against the same fixture and asserts the opposite outcome, which is what a genuine reversal of a rule looks like in a test suite.

The eight new cases cover the three bugs and the risk: the stage staying shut while the Tax Order of Payment is unsettled, staying open on a returned filing, the un-refunded fee, the sheet reachable after payment, release on the second payment settling, release through the ordinary path when the balance clears first, the outright refusal of `approveAndIssue` on a filing that owes money, and — in the browser — `a locked Apply stays reachable, and refuses to do anything`.

2. Renewal: naming the permit being renewed

What changed, and why

A renewal is a filing that says “renew this”. Until this commit, seven of them in the register did not say *which*.

Three things were done about that. The identification system was settled and written down. The renewal filing was made to name its permit, with an explicit way out for the applicants who cannot. And the permit chain — which permit succeeded which — was made a stored fact instead of something reconstructed from dates.

A defect in the business chooser was found on the way, and is described at the end.

The identification system

The client asked us to settle what identifies what. Three numbers, three jobs:

Identifier	Format	Answers	Lifetime
Business account no. (`businesses.ban`)	`BP-2026-0001`	which business?	permanent, across years
Permit number	`MCB-2026-000001`	which permit is being renewed?	one permit
Tracking ID	`BIZ-2026-00473`	where is my filing?	one application, in flight

All three formats are confirmed in `api/app/Support/Numbering.php` — the tracking id at line 33 (`BIZ-%d-%05d`), the permit number at line 42 (`%s-%d-%06d`), the account number at line 82 (`BP-%d-%04d`).

The permit prefix is per permit *type*, read from `permit_types.permit_number_prefix` and passed in at `WorkflowService.php:1118`. The live set is `BUSINESS → MCB`, `SANITARY → MCS`, `FSIC → MCF`, `OCCUPANCY → MCO`, `CEC → MCE`, `MARKET → MCM`, `ZONING → MCZ`. So an `MCB` number is a Mayor’s / Business Permit specifically.

One correction. “One permit, one type, one year” describes how the system is used, not a rule it enforces. The only database constraint on a permit number is that it is unique (`2026_07_24_000041_create_permits_table.php:13`); there is no composite unique on business, type and year, and the sequence runs globally per prefix per year rather than per business. Nothing today creates a second live permit of one type for one business in one year, but nothing stops it either. If BPLO’s rule is genuinely one-per-year, that is a constraint worth adding, and it would be cheap.

BAN became BP

The account number used to be formatted `BAN-2026-0001`. BizTrack has a blacklist feature, so `BAN-` on an owner’s own record read as a notice that they had been banned. Every number was rewritten keeping its sequence (`2026_08_30_000000_rename_ban_prefix_to_bp.php:44-47`), and the database column is still called `ban`, so nothing downstream had to change.

A correction on the count. The commit message says 718 numbers were rewritten. The migration uses a raw table update with no soft-delete filter, so it actually rewrote **779** rows — every business in the register. 718 is the count of *live* businesses; the other 61 are soft-deleted. The migration is right to include them: `Numbering::ban()` uses `withTrashed()` when it works out the next sequence number, so leaving the deleted rows on the old prefix would have corrupted the sequence.

The seven orphan renewals

At the time the work was done, 749 of 756 renewals in the register carried a prior permit and 7 did not. Checking against the register today, it has grown to 758 renewals, 751 with a prior permit — and the same 7 without. The orphans are stable; they were not a continuing leak, they were a set of historical rows.

Three routes in, one root cause: **a null was accepted as an answer without anyone ever having to give it.**

Filing	Status	Permits the business held	How
1	approved	2	written directly by `DemoSeeder`
7, 3367, 3382, 5008, 5011	draft	0	the question was never asked
3391	draft	3	the question was skipped past

The five are the interesting case. The picker's escape hatch was the *absence* of the question — the old wizard read, at `ApplyWizard.tsx:1397`:

```
: applicationType === 'renewal' && permits.length > 0 && permitId === null
```

and at line 1478, where a business held no permits, it rendered a static paragraph and no control at all. The comment above it was honest about why (line 1385): *"`permits.length > 0` is the escape the client's year-one applicants live on."* The intention was right. The implementation made the escape unrecorded.

Because afterwards, a null nobody was shown and a null somebody chose are the same row. The register could not tell "my permit was issued on paper" from "nobody asked me", and those need different handling: the first is a normal year-one renewal, the second is a filing that should never have been accepted.

The fix: an answer you give

`applications.prior_permit_declared_none` — a boolean, default false, `2026_08_30_000020_make_the_renewal_chain_a_fact.php:81–90`.

The escape is now a radio option in the same group as the permits (`ApplyWizard.tsx:1601–1631`), rendered as the last child of the same list. Its label depends on whether there is anything above it:

- with permits listed: *"None of these — my permit was issued on paper"*
- with no permits listed: *"This business has no permit issued through BizTrack"*

and either way a sub-label: *"Upload your paper permit under Documentary Requirements and we will carry on from there."*

Submit refuses a renewal or an amendment that carries neither (`ApplicationController.php:320–330`):

```
in_array($application->application_type, [ApplicationType::Renewal,
        ApplicationType::Amendment], true)
    && $application->prior_permit_id === null
    && ! $application->prior_permit_declared_none
```

> Say which permit you are renewing — pick it from your permits, or tell us this business has no permit issued through BizTrack.

(`renewing` becomes `amending` for an amendment; that is the only difference.)

Both writers resolve the contradiction the same way — if a permit is named, the "none" flag is cleared (`ApplicationController.php:190–191` , `PriorPermitController.php:76–82`). A permit picked is a stronger answer than a box ticked.

The permit itself is never typed. It is chosen from the business's own permits, delivered by the prefill endpoint, and cross-business ids are stripped when the applicant switches business (`ApplyWizard.tsx:1371–1375`). The server checks the same thing twice over, on create and on update, and refuses with *"The selected prior permit does not belong to this business."*

> **The escape must stay.** In year one, most renewals are renewals of permits the old counter process issued on paper. Removing the escape would trap exactly the people the system exists to serve. If BPLO would rather a business with no permit in the register filed as **new** instead, that is a smaller change than this one, not a larger — and it is logged as question A20b.

The permit chain is now a stored fact

`permits.prior_permit_id` : nullable, indexed, and deliberately **without** a foreign key (`2026_08_30_000020...:76–77`). The reason is in the migration: `permits` is self-referencing, and SQLite rebuilds the entire table to add a self-referencing constraint. On a register with real tester data in it that is a risk taken for very little — the column is written by one code path and backfilled by one statement, both of which check the target themselves.

The backfill is a correlated update with a same-business guard in the subquery and a `where prior_permit_id is null` so it can be re-run safely. Its figures were measured before the run and recorded in the migration's own docblock; we checked all five against the register and they are exact:

permits total	5,942
chained	2,722
null (the `new` filings)	3,220
links crossing to another business	0
permits linked to themselves	0

It is written by a `creating` hook on the model (`Permit.php:51–75`), not by the workflow service. The hook reads the application's `prior_permit_id` , bails out if the column is already set or there is no application, and writes only when the prior permit belongs to the same business. `approveAndIssue` does not touch the column at all — the hook is the only writer.

That placement is the point. A chain maintained at one call site is exactly the half-kept invariant that produced the seven orphans in the first place: correct wherever somebody remembered, silently absent everywhere else.

A correction. The reasoning was given to us as "`Permit::create()` has three callers". There are ten call sites; three are outside the test suite (`WorkflowService.php:1117`, and `DemoSeeder.php:115` and `:178`). The argument holds — the seeder mints permits directly and would have bypassed a service-level write — but "three callers" means three non-test callers.

A second, smaller inaccuracy is in the code itself: the docblock at `Permit.php:41–42` says "DemoSeeder and AnalyticsHistorySeeder both mint permits directly". `AnalyticsHistorySeeder` does not; it goes through `approveAndIssue()`. The conclusion is unaffected, but the comment names the wrong file.

Why this matters beyond tidiness

`RenewalOutcomes` decides whether a renewal was late. It does that by reconstructing the chain, and — this is worth being precise about — **it still does**. `RenewalOutcomes::cycles()` (`api/app/Support/RenewalOutcomes.php:313–378`) groups permits by `business_id : permit_type_id`, sorts by `valid_from` with the id as a tie-break, and treats consecutive pairs as a renewal, late if the gap exceeds `LATE_GRACE_DAYS` (which is 1).

That inference is a coin toss exactly where it matters most: on a business holding two permits of one type, where the sort order between them is decided by dates that may be identical. And the late/on-time label it produces is what trains the fitted renewal model used on the analytics screens.

So the honest statement is: **the column is written, and not yet read by the analytics it was added for.** The chain is now a fact, but `RenewalOutcomes` has not been switched over to use it. That is the follow-up piece of work, and it is now a small one — the data is there and verified.

A real defect in the business chooser

Found while testing the picker, and worth reporting in full because the first fix we tried was wrong in an instructive way.

The defect. The business chooser fetches one page of `PICKER_PAGE_SIZE` businesses — 200 (`web/src/lib/resources.ts:128`) — ordered newest-first. An owner past 200 businesses lost the oldest off the end. The oldest are precisely the ones whose permits are old enough to need renewing, so the chooser could render 200 businesses that cannot be renewed and drop the one that can.

The wrong fix, and why it was wrong. The obvious repair is to filter the list to businesses that hold a permit. It was tried, and it breaks the rule established one section above: *a business whose permits are on paper is not trapped, but must say so.* Filtering would remove from the chooser exactly the applicants the paper escape exists for — a business with a paper permit holds no permit in BizTrack, so it would vanish from its own owner's list. The wrong fix is documented in the code that replaced it, at `BusinessController.php:62–87`.

The shipped fix is ordering, not exclusion — `BusinessController.php:88–91`:

```
->withCount('permits')
->orderByRaw('CASE WHEN permits_count > 0 THEN 0 ELSE 1 END')
->orderByDesc('created_at')
->orderByDesc('id')
```

Permit holders sort first; businesses with paper permits stay reachable behind them. Nobody is excluded, and the row that matters is on the first page.

Two things this does not do, which we should say rather than let someone discover.

- **The 200 cap is still there.** Ordering makes it survivable, not gone. An owner with more than 200 permit-holding businesses would still lose the tail. The server supports paging — `page` is validated, `per_page` is clamped to 200, `meta.last_page` is returned — and the client never asks for page two. A paging helper, `businesses.page()`, exists in `resources.ts:224` and has zero callers. Nobody in the register is near that number today.
- **No test covers the ordering.** There is no assertion anywhere in the PHP or browser suites that permit-holders sort first. This fix is currently protected by nothing, and it is the kind of clause a later refactor of the list endpoint would drop without noticing.

A correction on the figure. We were told one demo owner holds 239 businesses of which 5 have a permit, and that number is also in the code comment at `BusinessController.php:69–71`. It is not reproducible in either database. In `database.sqlite` the largest owner holds 35 businesses with 1 permit; in `e2e.sqlite` the largest holds 261 with 6. The shape of the defect is real and lives in the end-to-end register — which grows on every run, which is why the exact pair drifted. The code comment should carry the shape, not a number that was true for an afternoon.

3. Amendment

What changed, and why

An amendment is a filing that alters an existing permit — a change of ownership, of location, or of the nature of the business. BizTrack could already record *what* was being amended. It could not record *which permit* was being amended.

That is the whole of the change here. An amendment now names its permit, the same way a renewal does, and the server refuses to accept one that names neither a permit nor a reason there is none to name.

Everything else in this section is an audit result rather than a change. We went looking for the amendment feature expecting to build it, found it already built, and are reporting what is there so that nobody builds it twice.

What was already working

Checked line by line, and all of it predates this commit:

Part	Where
Amendment columns writable on the model	`api/app/Models/Application.php:48–49`
Booleans cast, so the API returns `true` and not `1`	`api/app/Models/Application.php:77–80`
`has_amendments` computed on the server, never accepted from the browser	`api/app/Http/Controllers/Api/ApplicationController.php:582`
Validation when the application is created	`ApplicationController.php:161`
Validation when a draft is updated	`ApplicationController.php:230`
Exposed to the frontend as `amendments`	`api/app/Http/Resources/ApplicationResource.php:34–43`
Rendered on the officer's review sheet	`web/src/pages/officer/ReviewPage.tsx:1991–2019`
Tests	`api/tests/Feature/AmendmentDetailsTest.php` , 9 cases

The `has_amendments` point is worth spelling out because it is the kind of thing that looks like a detail and is not. The flag is derived at `ApplicationController.php:582` from the four answers:

```
'has_amendments' => $ownership || $location || $nature || $other !== '',
```

It is deliberately absent from the validation rules, so a browser cannot send it. A client that could set the flag itself could file an amendment that claimed to amend nothing, or claimed to amend something it had not filled in.

The test count did not change in this commit. The diff shows ten changed lines in `AmendmentDetailsTest.php`, but that is one existing test edited, not a test added — `it('submits once something is actually being amended')` at line 114 now carries a prior permit, because without one that filing no longer submits at all.

The paper form's "Amendment from:" block

`AmendmentState` in `web/src/pages/applicant/ApplyWizard.tsx:368–374` models the block as three tick boxes and one text field. `AMENDMENT_KINDS` at lines 388-392 holds **three** entries, not four:

```
{ key: 'ownership', label: 'Ownership' },
{ key: 'location',  label: 'Location' },
{ key: 'nature',    label: 'Nature of Business' },
```

"Others (specify)" is deliberately outside that list, because on the paper form you cannot tick Others without writing the other in. So typing **is** ticking: the text field is rendered on its own with no checkbox beside it, and a separate tick could only ever contradict the text. That rule holds in four independent places — the wizard's own submit gate (`ApplyWizard.tsx:1389–1390`), the rendering (`:1678–1689`), the server's derivation (`ApplicationController.php:579–586`), and the summary line on the model (`Application.php:98`, which prints `Others: <text>`).

There is no fifth boolean column for "others", and that is on purpose.

The gap that was real: naming the permit

The permit picker was already shown for amendments. What it was not, was required. The old gate read:

```
applicationType === 'renewal' && permits.length > 0 && permitId === null
```

Two ways past it: you were not a renewal, or your business held no permits. An amendment could therefore be confirmed and submitted naming nothing at all.

The rule is now the same for both filing types, on the server, in one place — `ApplicationController.php:320–330`:

```
throw ValidationException::withMessages([
    'prior_permit_id' => ["Say which permit you are {$verb} — pick it from your permits, or
        tell us this business has no permit issued through BizTrack."],
]);
```

`$verb` is "renewing" or "amending"; it is the only difference between the two cases. The escape hatch is the same one described in section 2: a radio option reading *"None of these — my permit was issued on paper"*, recorded on `applications.prior_permit_declared_none`, which has to be chosen rather than fallen through.

Two gaps reported rather than half-fixed

There is no application-form PDF anywhere in the system. We enumerated every document generator in the codebase. All of them use `barryvdh/laravel-dompdf` with views under `api/resources/views/pdf/`, and the complete list is:

1. the permit certificate (`PermitController::pdf`)
2. the payment receipt (`PaymentController::receipt`)
3. four analytics reports (dashboard, processing time, business growth, renewal risk)
4. one CSV export

There is exactly one `window.print()` in the whole frontend, at `web/src/pages/applicant/PermitDetailPage.tsx:289`, and it prints the permit certificate. There are no `@media print` rules in `web/src` at all.

So the "Amendment from:" block is captured, stored, validated and shown on screen, and printed nowhere — because *no* application form is printed. That is an absent feature, not a broken wire, and it is the same absence for a new application and a renewal. Building it is a separate piece of work, and it needs the paper form first (see section 4).

The applicant cannot see their own amendment block. The officer's review sheet renders it under the heading "Amendment From". The applicant's `ApplicationDetailPage.tsx` contains no reference to amendments at all — 797 lines, zero occurrences of the string. This asymmetry predates the commit; we are naming it rather than fixing it, because where that block should sit on the applicant's screen is a design question, not a bug fix.

One thing that page does handle well and is worth copying: the officer's sheet has a fallback for filings created before the amendment question existed —

> This filing predates the amendment question and does not record what is being amended. Ask the applicant through Messages before deciding.

which is the right answer to "the data is missing" on a register that has history in it.

Open question logged

A20c. Does an amendment also name the permit it amends?

Why it matters. An amendment alters one permit's record. "Amend my business" tells the counter no more than "renew my business" does when the shop holds three permits with three expiry dates. Checklist item 50 asked for the choice on renewals; we could see no reason it stops there.

What we assumed meanwhile. Yes — the same question, the same dialog, the same submit gate. If the paper amendment form has no permit-number box, this is one field too many and we take it out. That is a smaller change than the one we made, not a larger one.

4. Office forms: asking each question once

What changed, and why

The rule the client gave us is simple to state: **each office sheet asks only what its paper form needs, minus what the wizard already holds.** A field the applicant has already answered must not be asked again. Where the office genuinely needs it on their form, it should be carried over and shown — not re-collected.

We audited all six office sheets against that rule. One real duplicate was found, and it was the damaging kind. Ten other fields that look like duplicates were checked and left alone, and this report lists them so nobody has to check them twice.

How the rule is enforced

Carry-over is not a convention that a developer has to remember. It is a function — `OfficeFormController::withDerived()`, lines 220-386 — whose last line is:

```
return $derived + $formData;
```

That is a PHP array union, in which the **left** operand wins. A derived answer overwrites whatever the browser sent for that key. And it runs on both paths: when the sheet is read (line 67) and again on every write (line 156). So a client cannot supply its own value for a carried-over field, even by trying.

The comment above it puts the rule in one sentence: *"Overlay the answers the system already holds on top of a payload. These always win: the paper form still carries them, but nobody types them."*

The fields currently derived are `application_date` (every sheet), `application_type` (Zoning, Sanitary, CEC, Market), `total_floor_area_sqm`, `building_storeys`, `site_tenure`, `authorized_representative` (Zoning), `workers_requiring_health_certs` (Sanitary) and `certificate_applied_for` (Fire).

Two details of the implementation are worth defending, because they look like extra work:

- **Derived answers are shown, not silently carried.** They render read-only, tagged *"(from your application)"*. The reason is in the code: the applicant signs a statutory declaration on these sheets, and *"carrying them silently is worse than re-asking: the applicant signs a statutory declaration without ever seeing what it says about them."*
- **The sheet is synthesised on read**, even if it has never been saved, so the carried answers are visible the first time the applicant opens the form rather than appearing after the first save.

The precedent: capitalization

This is the fault we are guarding against, and it has already happened once.

Capitalization used to be asked twice — once against each business line in the picker, and again in Business & Tax Profile — and only the second answer was ever assessed. So the two drifted the moment anyone edited the first, and the one the applicant could see on the earlier screen was not the one they were billed on.

It was removed on 5 August (commit `97eb5ab`, *"fix(apply): ask for capital once, and say what the category actually is"*), and the tombstone is still in the wizard at `ApplyWizard.tsx:275–281`:

> Across 400 filed applications not one pair disagreed, which is what one question asked twice looks like.

Where that figure comes from. It was measured against the live register on 5 August and recorded in prose, in the commit message and the code comment. There is no fixture, test or seeder that asserts it, so it cannot be re-derived from the repository alone, and the register has grown since. We are repeating it as what it is: a measurement taken once, not a standing claim.

The one real duplicate, and why it was the bad kind

The **Sanitary** sheet asked *"No. of Workers Requiring Health Certificates"* in a free-text box (`OfficeFormStep.tsx`, lines 765-769 of the previous version).

Nothing read it. Searching the whole repository at the parent commit for `workers_requiring_health_certs` returns exactly two hits — the input's own `value` and its `onChange`. Zero hits in the API, zero in the fee engine, zero in the officer's review sheet, zero in the tests.

Meanwhile the health certificate fee bills a different number. It is **₱50 per employee per year** under **Sec. 4D.02** of the Revenue Code, and its basis is `fee_profile.employees` — the headcount from Business & Tax Profile.

> **A correction.** We were told this fee is computed in `api/app/Support/PermitFees.php`. It is not; that file contains no fee logic at all — it is 118 lines holding the payment ledger helpers (`balance()`, `hasClearedPayment()`, `hasOutstandingBalance()`). The fee is a seeded rule in `api/database/data/revenue_code/sanitary_garbage.json`, lines 579-603, with `"section": "Sec. 4D.02"`, `"unit_amount": 50`, `"unit_key": "employees"` and a condition on the `employees_need_health_certificates` flag. It is evaluated by `api/app/Services/FeeCalculator.php` at lines 219 and 242. The rate, the section and the basis are all as described; only the file was wrong.

So this was not a cosmetic duplicate. Capitalization's two answers merely drifted. These two land **side by side on one filing**: a headcount printed on the City Health Office's own sheet, and a Tax Order of Payment computed for a different headcount, for that same fee. An officer reading "4 workers" beside a bill for five is looking at a discrepancy the applicant never made and cannot explain.

The box is now derived and read-only (`OfficeFormController.php:345–356`):

```
$derived['workers_requiring_health_certs'] = match (true) {
    ! $needsCertificates => 'None',
    is_numeric($employees) => (string) (int) $employees,
    // Flagged but no headcount: the profile is half-filled, so say
    // nothing rather than print a zero the office would act on.
    default => '',
};
```

Three cases, and the third is the careful one. When the applicant has said no employee needs a certificate, it prints **"None"** — an answer, not a blank that looks like an omission. When the flag is set but no headcount exists, it prints **blank rather than `0`**, because a zero on a City Health form is a statement the applicant did not make and the office might act on.

The screen says where the number came from: *"From the employee count on your Business & Tax Profile — the same number the health certificate fee is charged on."*

Locked by three tests in `api/tests/Feature/OfficeFormTest.php` — line 118 (a posted value is overwritten by the profile's), line 137 (`'None'`), line 149 (the empty string).

Checked, and genuinely distinct

These ten look like duplicates and are not. Each is listed with the reason, so that the next audit does not repeat the work.

Sheet	Field	Why it stays
Zoning	`zoning_project_description`	A free description of the project as CPDO assesses it. The wizard holds a line of business (a PSIC code), which is not the same statement
Zoning	`zoning_industrial_project_type`	Pollutive / Non-Pollutive / Hazardous / Non-Hazardous / Other. Nothing in the wizard classifies a business this way
CEC	`owner_birthday`	**No birthdate exists anywhere in the schema.** An exhaustive search of every migration for `birthday`, `birthdate`, `birth_date`, `date_of_birth`, `dob` and `born` returns zero hits, and neither `User` nor `BusinessOwner` has such a field. It lives only inside this sheet's JSON
Occupancy	`building_permit_no`	A number issued by another office, on another document
Occupancy	`fsec_no`	Likewise
Occupancy	Full vs partial occupancy	See below — this one is explicitly protected
Market	`market_name`	Which public market. The wizard holds an address, not a market
Market	`stall_no`	Likewise
Sanitary	`water_source`	A premises fact the wizard never asks for
Sanitary	`sanitary_classification`	The CHO's own classification of the establishment

Two of the key names differ from how they were described to us: the Zoning fields are `zoning_project_description` and `zoning_industrial_project_type`, not the bare `project_description` and `project_type`.

The Occupancy full-vs-partial choice deserves its own note, because it is not merely "left alone" — it is actively defended. It is stored under the key `application_type`, which is the *same key* that is derived from the filing (new / renewal / amendment) on four other sheets. The controller excludes Occupancy from that derivation by name, with the reason written beside it (lines 382-383): *"OCCUPANCY's own `application_type` is Full vs Partial occupancy — a real applicant decision, not the new/renewal the system already knows."* A test at `OfficeFormTest.php:161` holds it there.

That is a collision that would have quietly overwritten an applicant's answer with an unrelated one, and it is the strongest argument in this section for doing the audit at all.

The honest limitation

We do not have all the paper forms, and every assumption made where the paper is unknown is written down. But the blanket statement *"we do not have the paper forms"* is broader than the truth, and the report should be precise:

- **Four sheets were transcribed from the real document.** `OfficeFormStep.tsx` lines 20-26 records it: *"The CHO, CENRO, BFP and OBO forms were always copied from the document the counter hands out, field label for field label, so what BizTrack asks is what the counter asks, word for word."*
- **The zoning form has been received.** `docs/questions-for-malabon.md` §E4 is marked ANSWERED: *Application for Locational Clearance (Business Activities)*, document ID **MCG-CPDD-FO-003**, version **1.2**, effective 01-09-2026.
- **One sheet has no paper behind it at all** — Market. Its own metadata says so: `'Interim form — the office has no printed version of this application'`.

The three BPLO form references the wizard is built against are real and exact, in `ApplyWizard.tsx:265–269`:

```
new:       { title: 'Application for New Business Permit',        ref: 'MCG-BPLO-FO-001 · v2.0' },
renewal:   { title: 'Application for Renewal of Business Permit', ref: 'MCG-BPLO-FO-002 · v2.0' },
amendment: { title: 'Application for Amendment of Business Permit', ref: 'MCG-BPLO-FO-003 · v2.0' },
```

and the BPLO item numbers recorded in field comments — A5, A6, A9, A13/A14/A15, B6, and B8 (B7 on the renewal form) — are all present. All of them are in `ApplyWizard.tsx`; none are in `OfficeFormStep.tsx`, which is consistent with the item numbers belonging to the BPLO form rather than to the office sheets.

A dangling citation we found and did not fix. `OfficeFormController.php:336–337` supports the health-certificate assumption by citing *"docs/questions-for-malabon.md §E — the CHO paper form has been requested and not received"*. Section E lists nine documents (E1-E9) and the CHO form is not among them. The citation points at nothing. The question is real, but it lives at **A4b**, not in §E — and §E should probably gain an entry for the CHO form, since the sheet was transcribed from a counter handout rather than from a controlled copy.

The three questions logged

Added to `docs/questions-for-malabon.md` by this commit, in the house style: the question as a heading, the facts as they stand, **Why it matters**, and **What we assumed meanwhile**.

A4b. On the CHO sheet, is "No. of Workers Requiring Health Certificates" the same headcount we bill? — with a fourth beat, because a wrong assumption here costs twice: if CHO means a narrower subset (the office staff of a food establishment excluded, say) then it is a genuinely separate question we owe them *and* the fee is billing the wrong headcount today.

A20b. And when the business has no permit in BizTrack? — the paper-permit escape has to be ticked, not fallen into.

A20c. Does an amendment also name the permit it amends? — if the paper amendment form has no permit-number box, this is one field too many and we take it out.

Two small corrections about the logging itself. **A20b is not a peer entry** — it is bold text inside the body of A20, so it does not appear in a heading index the way A4b and A20c do. And the house style has **two** recurring labels, not three: `**Why it matters.**` appears on 58 of the 66 question headings, and `**What we assumed meanwhile.**` on 20 — it is used only where something has actually been shipped on a guess, which is the correct use.

5. Messaging: a conversation now has an addressee

What changed, and why

Until this commit, a message in BizTrack was addressed to a *filing*. It was not addressed to anybody.

`message_threads` had no department column at all. The columns were `id`, `application_id`, `subject`, `status` and the two timestamps — confirmed against the original create migration (`api/database/migrations/2026_07_24_000060_create_message_threads_table.php:11–15`), the later column addition (`2026_07_24_000073_align_tables_with_manuscript.php:105–108`), and the schema of an August snapshot database.

That absence matters more than it sounds. “Send this to the correct office” was not a rule the system was enforcing badly — it was not a concept the system could express. One filing had one conversation, everybody who could see the filing wrote into it, and who a given message was *for* was something a reader had to infer from its contents.

A thread is now `(application, department)`. An owner may write to any office actually handling their filing, plus BPLO. An officer sees their own office’s conversations and no others. Both rules are enforced on the server, not hidden in the browser.

The migration

`api/database/migrations/2026_08_30_000010_scope_a_message_thread_to_an_office.php`

- **Adds the column** (line 77-81): `$table->foreignId('department_id')->nullable()->after('application_id')->constrained()->restrictOnDelete();` `restrictOnDelete` rather than cascade: deleting a department should fail loudly while conversations reference it, not silently take 520 threads with it.
- **Backfills to BPLO by code, not by id** (line 90-95): `$bplo = DB::table('departments')->where('code', 'BPLO')->value('id');` guarded by `if ($bplo !== null)`. A hard-coded `1` would be right on our machine and wrong on the city’s.
- **Swaps the unique index** (line 98-99): drops `message_threads_application_id_unique`, adds `message_threads_application_id_department_id_unique`. This is the change that makes six conversations on one filing legal; without it the second office to be written to would collide with the first.

A correction to how this was described to us. The column was made nullable and then backfilled, on the stated reasoning that a `NOT NULL` column forces SQLite to rebuild the table, and the register holds real tester data. The first half of that is true and the invariant is genuinely held above the schema (see below) — but the table was rebuilt anyway. Adding the foreign key did it. Comparing the stored schema before and after, the *pre-existing* `application_id` foreign key text changed from

```
references "applications"("id") on delete cascade
```

to

```
references applications("id") on delete cascade on update no action
```

An in-place `ALTER TABLE ADD COLUMN` cannot rewrite another column's foreign key clause; that rewrite is Laravel re-deriving the constraints during a create-copy-drop-rename. No data was lost and the outcome is correct, but "nullable avoids the rebuild" is not what happened. If we had wanted to avoid the rebuild we would have had to skip the foreign key too, which is not a trade worth making.

The invariant is held in the model instead — `api/app/Models/MessageThread.php:37–44`:

```php
protected static function booted(): void
{
    static::creating(function (self $thread) {
        if ($thread->department_id === null) {
            $thread->department_id = Department::where('code', 'BPLO')->value('id');
        }
    });
}
```

So a thread created without a department gets BPLO, whatever created it.

Row counts

Measured directly against `api/database/database.sqlite` after the migration:

	count
`message_threads`	520
`messages`	2,094
`message_attachments`	0
threads with a null department	0
threads grouped by department	`BPLO` — 520, and nothing else

The "before" side of those figures cannot be measured directly — there is no pre-migration snapshot of that file — so the honest statement is: the migration contains no `DELETE` and no `INSERT`, and the August 7 snapshot (`api/database/e2e-final.sqlite`) holds 519 threads and 2,088 messages, which is consistent with 520 → 520 and 2,094 → 2,094 across three weeks of ordinary use. The `e2e.sqlite` copy also migrated with zero null departments.

Two questions, deliberately kept apart

This is the part worth reading twice, because conflating these two questions is how permission bugs get written.

May you READ a conversation?

`api/app/Support/ApplicationVisibility.php:166–169`:

```php
public static function readsThreadOf(?User $user, ?int $threadDepartmentId): bool
{
    return self::readsOfficeSheet($user, $threadDepartmentId);
}
```

It delegates rather than deciding. `readsOfficeSheet` (lines 91-106) is the same predicate that already governs office forms, issued clearances and inspection findings — four call sites, one function. If the rule for who may read a sanitary sheet is right, the rule for who may read the sanitary conversation is right too, and it cannot drift out of step with it.

The branches:

Reader	Result	Line
No user	false	93-95
BPLO, super admin — anyone with `application.view_any_office`	every conversation	96-98
Applicant	every conversation on their own filing	100-102
Officer with a department	only their own department's	104-105
Reviewer with **no** department	nothing	104-105, fail-closed

One point of precision the code deserves: the applicant branch literally tests "is not a reviewer" (`! $user->hasPermission(self::VIEW_ALL)`), not "owns this filing". Ownership is enforced one door earlier, by `canView()` (lines 237-252) via `MessageController::authorizeParticipant()` (lines 810-817). The effect is what we describe, but the ownership check is not in this function.

May you ADDRESS an office?

A different question with a different answer: every department holding an `ApplicationAssignment` on that filing, **plus BPLO** — because BPLO coordinates and an applicant must always be able to reach the front desk.

`MessageController::addressableOffices()` (lines 94-108) builds the set from the filing's assignments and then adds BPLO if it is not already there, resolving it by code (`Department::where('code', 'BPLO')`) rather than by id.

Refused server-side on both endpoints, with the same status and the same sentence:

- **POST** a message — `resolveAddressee()` , lines 182-186
- **GET** the transcript with `?department_id=` — `index()` , lines 627-631

```
abort_unless(
    $office !== null && ApplicationVisibility::readsThreadOf($user, $office->id),
    403,
    'That office is not handling this application, so it cannot be messaged about it.'
);
```

Hiding the option in the UI would have been easier and would not have been a rule. The tests check the refusal, not the absence of a button:

- `it('refuses an applicant writing to an office that is not on their filing')` — `OfficeScopingTest.php:612` , asserts 403 on **both** the POST and the `GET ?department_id=` , then asserts the two legitimate offices are accepted
- `it('will not let one office post into another office\'s conversation')` — line 655
- `it('offers an applicant only the offices actually on their filing')` — line 641

- `it('shows an officer only its own office among a filing\'s conversations')` — line 673; CHO sees `['CHO']`, BFP sees `['BFP']`, BPLO sees `['BFP','BPLO','CHO']`

Two latent bugs went with the rewrite

Both are worth naming, because neither was reported by anyone and both are the kind that get discovered by an auditor rather than a user.

BPLO's coordinating turns were invisible to the offices being coordinated. The old code said so itself, in its own docblock (lines 60-66 of the previous `MessageController`):

> Deliberately left OUT, and this is the visible trade: BPLO's and the super admin's turns are hidden from a scoped office too. BPLO is another office by this rule and the client named no exception for it.

It was a known trade, honestly recorded, and it was the wrong one. The office whose job is to coordinate the other six was the one office whose messages the other six could not see.

The old boundary keyed on the sender's *current* department. Old `MessageController`, lines 95-100:

```
if ($deptId) {
    $q->orWhereExists(fn ($sub) => $sub->selectRaw('1')
        ->from('users as vu')
        ->whereColumn('vu.id', 'messages.sender_user_id')
        ->where('vu.department_id', $deptId));
}
```

`users.department_id` is a live column. Move an officer from CHO to BFP and every message they had ever sent moved with them: their old CHO turns disappeared from CHO's view and appeared in BFP's. The new query joins `message_threads as vt` on `vt.department_id` instead (lines 229-232) — a fact fixed at the moment of sending, which is what an audit trail has to be.

The assumption with a visible cost

All 520 historical threads were backfilled to BPLO, because that is where they came from and because BPLO is the fail-closed answer. The consequence is immediate and will be noticed on a demo: a scoped office no longer sees its own historical turns. Log in as `sanitary@biztrack.local` (the demo account seeded at `DemoSeeder.php:52`, City Health Office) and the message inbox is empty of history, because `readsThreadOf(sanitaryOfficer, bploId)` is false for every one of those 520 threads.

The migration records the cost itself, at lines 52-57: *"a historical turn written by, say, the fire inspector now lives in a BPLO thread, so the fire office no longer sees its own old words."*

We think this is the right direction to fail — showing an office somebody else's correspondence is a worse mistake than showing it too little of its own — but it is a change in visible behaviour and should not surprise anyone mid-demonstration.

> **Open question.** If BPLO would rather the history were distributed to the offices that wrote it, that is a second backfill: for each thread, resolve the department from the messages inside it. It is doable, it is not free, and it would be guessing on threads where two offices both wrote.

No new routes

The routes did not change — `git diff 9a33fe6^ 9a33fe6 -- api/routes/` is empty. The addressee arrives as an optional `department_id` parameter on the endpoints that already existed (`api/routes/workflow.php:113–116`), validated at `MessageController.php:616` for the GET and `:708` for the POST.

The list of offices you may write to rides on `meta.offices`, produced by `officeRowsForFiling()` (lines 675-691) and attached to the transcript's meta at line 640. Each entry carries `department_id`, `code`, `name`, `thread_id`, `messages_count`, `last_message_at` and `can_message`, sorted busiest-first with silent offices last by name. The frontend reads it at `web/src/components/MessagesPanel.tsx:304` and `can_message` drives the closed-conversation state.

One correction: `meta.offices` is exact for the transcript endpoint, but the **inbox** endpoint carries the same array one level down, on each row as `data[].offices` (`threadRow()`, line 434). Same shape, different place.

Tests

Eight cases added across two files, and several existing ones rewritten in place (so the diff is larger than the net count):

- `api/tests/Feature/MessageThreadsTest.php` — 6 → 9
- `api/tests/Feature/OfficeScopingTest.php` — 28 → 33

New names include `it('names the offices an applicant may talk to on each inbox row')`, `it('counts each office\'s conversation separately on the inbox row')`, and `it('names the office every message turn belongs to')`.

6. Known gaps and operational notes

None of these are in the five workstreams above. They are things the work uncovered, each one small enough to name and too consequential to leave unnamed.

An assignment can be approved twice, and it moves an RA 11032 figure

`AssignmentController::approve` validates the remarks and checks the officer's department, then hands off to `WorkflowService::approveAssignment` (`WorkflowService.php:565–581`). That method guards the **application's** terminal status — and never the **assignment's**. Its own docblock says so, and explains why the guard is where it is:

> The guard is on the FILING, not on the assignment, and that distinction is the bug (INS-5).

So an assignment that is already `Completed` can be approved again over the API. It is unreachable from the interface — the button is not rendered on a completed assignment — but the endpoint accepts it, and the effect is not harmless. Line 577 re-stamps `completed_at`, and `assigned_at → completed_at` is exactly the interval that four different pieces of analytics measure an office's service time on: `ProcessingTimeAnalytics`, `DashboardAnalytics`, `StaffingSimulation` and the SPC control charts.

A re-approval therefore silently resets that office's RA 11032 service-time record for that filing, and nothing in the audit log distinguishes the second stamp from the first. Left as its own piece of work: the fix is a guard, but choosing between "refuse" and "no-op" is a decision about what a duplicate approval *means*, and it should be made deliberately.

The end-to-end stack copies the database and never migrates it

`web/scripts/e2e-stack.sh` copies `api/database/database.sqlite` to a throwaway file and serves the copy. It never runs `php artisan migrate` against the copy.

The consequence is conditional, and we should be exact about it rather than alarming: the copy inherits whatever schema the live database has. We raised a fresh stack while writing this report and checked — all three of this commit's migrations show as `Ran` on the copy, because the live database is itself at head, and the suite ran against it without trouble.

The failure mode arrives when the live database is behind the code: every new stack then 500s on the new columns until somebody migrates the copy by hand. That is a real trap, it will be sprung by whoever next pulls this branch onto a machine with an older register, and the fix is one line in the script.

Printed permits carry a verify link nobody outside can follow

`api/.env` sets `FRONTEND_URL=http://localhost:5173`. That value becomes the `verify_url` on every permit, in the API resource (`PermitResource.php:33`) and on the printed certificate (`PermitController.php:268`).

So a permit printed from the live demo carries a QR/verify address that resolves only on the machine that generated it. Anyone on the tunnel — an adviser, a panel member, a BPLO officer — gets nothing. The file is not in version control and is environment-specific, which is why it has survived; it needs setting to the tunnel address before any demonstration where a permit gets printed.

Refunds are not modelled

Covered in section 1, repeated here because it is the one item on this list that needs an answer from City Hall rather than a decision from us. A clearance withdrawn after its fee has been paid leaves the filing in credit, the balance floors at zero, and no screen offers the money back.

Two smaller ones

- **The business-chooser ordering has no test.** The `CASE WHEN permits_count > 0` clause that keeps renewable businesses on the first page is asserted nowhere in either suite. It is the kind of clause a later refactor drops silently.
- **`permits.prior_permit_id` is written but not yet read** by the analytics it was added for. `RenewalOutcomes` still infers the chain from `(business, permit_type, dates)`. The data is now there and verified; switching the inference over is a small, separate change.

7. Where this report differs from the brief we were given

Every claim in the brief was checked against the code, the migrations, the tests and — where a figure was about the register — the database. Most held. These did not, and the report above states the corrected version in place.

Claim as given	What we found
The release gate was *misplaced* in `approveAndIssue()` and has now been split	Relative to the immediate parent commit, the gate did not exist at all; the parent's docblock said there should not be one, because under one-payment there was never a balance. The misplaced-gate bug belongs to the pre-4-August build of this design. This commit re-adds it, already split
Withdrawing a paid-for clearance drops the balance *below* what was received	The balance cannot go negative — `PermitFees::balance()` floors at `0.0`. The filing is left in silent credit instead, showing Paid greater than Assessed and "Balance due ₱0.00" in green. The risk is real; the mechanism is different, and arguably worse for being invisible
Ten tests renamed, seven new	16 renamed and 8 new across the two files (12 + 7 in `ClearanceStageTest.php`, 4 + 1 in `clearances.spec.ts`). "Seven new" is right for the PHP file alone
Applying for a clearance after release is "allowed by the data model but not surfaced on the screen"	It is surfaced. The stage link renders for every non-draft status including Approved. The behaviour is right; the code comment is stale
A rejected clearance does not kill the business permit	True as a decision, but not enforced and not currently reachable: `AssignmentStatus` has no `Rejected` case at all. The API still advertises `'rejected'` as a clearance state that nothing can produce
The BAN→BP rename rewrote 718 numbers	It rewrote **779** — every business including the 61 soft-deleted ones, which it had to, because the sequence generator counts trashed rows. 718 is the live-business subset
756 renewals, 749 with a prior permit, 7 without	Correct when measured; the register has since grown to **758** / **751** / **7**. The same seven orphans, and no new ones
A renewal is one permit, one type, one year	A convention, not a constraint. The only database rule is that `permit_number` is unique — no composite unique on business, type and year
`Permit::create()` has three callers	Ten call sites; **three outside the test suite**. The argument for the model hook is unaffected. (A related code comment names `AnalyticsHistorySeeder` as a direct creator; it is not — it goes through `approveAndIssue`)
One demo owner holds 239 businesses of which 5 have a permit	Not reproducible in either database: the largest owner in `database.sqlite` holds 35 with 1 permit; in `e2e.sqlite`, 261 with 6. The defect is real and lives in the end-to-end register, which grows on every run. The figure in the code comment should be a shape, not a number

Claim as given	What we found
`message_threads` was nullable-then-backfilled to avoid a SQLite table rebuild	The table was rebuilt anyway, by the foreign key. The give-away is that the *pre-existing* `application_id` FK clause was rewritten, which `ADD COLUMN` cannot do. No data was lost; the stated reason is not what happened
The health-certificate fee is computed in `PermitFees.php`	`PermitFees.php` contains no fee logic at all. The fee is a seeded revenue-code rule in `sanitary_garbage.json:579–603`, evaluated by `FeeCalculator.php`. The rate (₱50), the section (Sec. 4D.02) and the basis (`fee_profile.employees`) are all as stated
`AMENDMENT_KINDS` models four kinds including "Others (specify)"	It holds three. "Others" is deliberately outside the list because on the paper form the text *is* the tick, and a separate checkbox could only contradict it. `AmendmentState` is the four-field type
The amendment fix reuses the same dialog and mechanism as renewal	The dialog was already shared and already rendered the permit picker for amendments. The defect was that it was *optional* for amendments — the gate read `applicationType === 'renewal' && …`. What is new is `prior_permit_declared_none` and a submit gate that genuinely covers both types
Zoning fields `project_description` / industrial `project_type`	The keys are `zoning_project_description` and `zoning_industrial_project_type`
We do not have the paper forms	Overbroad. Four sheets (CHO, CENRO, BFP, OBO) were transcribed from the counter's document field-by-field, and the zoning form has been **received** (§E4, `MCG–CPDD–FO–003 v1.2`). Only the Market sheet has no paper behind it, and it says so in its own metadata
Three new questions logged	Three subjects, yes — but A20b is bold text inside A20, not a heading of its own, so it does not appear in a heading index alongside A4b and A20c. Section E is also cited by `OfficeFormController.php:336` for a CHO form that §E does not list
The house style has three parts	Two recurring labels: `**Why it matters.**` (58 of 66 entries) and `**What we assumed meanwhile.**` (20 of 66, used only where something was shipped on a guess)
The test suites take about six minutes each	The backend suite takes **85 seconds**
`docs/clearances-after-payment.md` is the spec	It is what the code was built against and is cited by roughly fifteen code comments — but the file still opens with `# SUPERSEDED — 4 August 2026` and refers the reader to the design this commit reversed. Neither clearance document was touched by this

Claim as given	What we found
	commit, so the repository now holds two specifications that each claim to supersede the other

Two figures in the brief were exactly right and are worth saying so, because they were the ones most worth checking: the backfill measurements (**2,722 chained / 3,220 null / 0 cross-business / 0 self-links**, against 5,942 permits) and the messaging row counts (**520 threads / 2,094 messages / 0 attachments / 0 null departments**).